

# Outlines

- Introduction to operator overloading
- Overloading unary operators
- Overloading binary operators
- Overloading unary and binary operators using friend function
- Overloading I/O operators

# Operator Overloading - Introduction

The meaning of an operator is always same for the variables of basic types:

- Like: int, float, double, etc.
- Eg: To add two integers, + operator is used.

However, for user-defined types (like, objects, structures) we can redefine the way operator works:

- Like concatenating string data members of two objects of same class.

This feature in C++ that allows programmers to redefine the meaning of an operator with a special meaning for a user defined data types is known as **operator overloading.**”

# Recall Function Overloading

Function Overloading allows multiple functions to have the same name but different parameters. The correct function is chosen based on the arguments passed.

```
int add(int a, int b) {  
    return a + b;  
}  
  
double add(double a, double b) {  
    return a + b;  
}  
  
int main() {  
    cout << add(2, 3) << endl;        // Calls int version  
    cout << add(2.5, 3.5) << endl;    // Calls double version  
    return 0;  
}
```

**In C++, operators are implemented as functions.**

- In C++, operators like **+**, **-**, **\***, **==** are implemented as special functions.
- These operator functions can be overloaded just like regular functions.

**By using function overloading on the operator functions, we can define our own versions of the operators that work with different data types (including classes).**

- By overloading operator functions, we can create custom behavior for operators when they are used with user-defined types like classes.
- This makes it possible to use operators intuitively with custom objects.

## An Example:

```
// The same operator + performs different tasks on different data types
int a = 4, b = 5;
cout << a + b << endl; // int + int // Addition
string h = "Hello", w = "World!";
cout << h + w << endl; // string + string // Concatenation
```

The compiler comes with a built-in version of the plus operator (+) for operands of basic data types.

Expression **a + b** becomes function call **operator+(a, b)** (**operator+** is the function name).

Similarly, **h + w** becomes function call **operator+(h, w)**.

**Depending on the type of the arguments passed, the compiler will make a call to the appropriate overloaded function.**

# Resolving overloaded operators

The process of selecting the most appropriate overloaded function or operator is called overload resolution.

When evaluating an expression containing an operator, the compiler uses the following rules:

- If all of the operands are fundamental data types,
  - The compiler will call a built-in routine if one exists. If one does not exist, the compiler will produce a compiler error.
- If any of the operands are user-defined data types (e.g. one of your classes),
  - The compiler looks to see whether the type has a matching overloaded operator function that it can call. If it can't find one, it will try to convert one or more of the user-defined type operands into fundamental data types so it can use a matching built-in operator. If that fails, then it will produce a compile error.

# Why Operator Overloading?

**Redefine Operator Behavior:** Operator overloading allows us to redefine the way operator works for user-defined types only.

**Closer to Target Domain:** Operator overloading enables us to use notation closer to the target domain.

- We can add two matrices by writing  $M1 + M2$  rather than writing  $M1.add(M2)$ .
- We can replace codes like `add(multiply(a, b), divide(a, b));` with  $(a*b) + (a/b)$

**Flexibility:** If we find ourselves limited by the way the C++ operators work, we can change them to do whatever we want.

# Operator Overloading

## A Form of Polymorphism:

- Operator overloading allows operators to have different behaviors based on their operands, enabling polymorphism for user-defined types.

All operators can be overloaded except

- Class member access operator (., .\*)
- Scope resolution operator(::)
- Size operator (sizeof)
- Conditional operator ( ? : )
- typeid, preprocessor symbols (#, ##) and
- the casting operators (e.g., static\_cast))

# Operator Overloading

Syntax:

*returnType* **operator\*** (*parameters*) ;

↑                    ↑            ↑

*any type*            *keyword*   *operator symbol*

- **Keyword:** *operator*
- **Symbol:** The operator symbol to be overloaded

## Methods of Overloading Operators:

1. Using Member Functions
2. Using Friend Functions
3. Using Normal Functions

## Operator functions as member functions

- An overloaded operator always requires one less argument than its number of operands. This is because the object on which the operator is called acts as an implicit operand.
- A member function has no arguments for unary operators and one argument for binary operators

## Operator functions as non-member functions

- Must be friends if needs to access private or protected members
- Functions require one argument for unary operator and two arguments for binary operator

# Overloading Unary Operators

## Unary Operators:

- Unary operators act on a single operand.
- Examples include increment (**++**), decrement (**--**), unary minus (**-**), and logical NOT (**!**).

Basic syntax for overloading ++ operator:

```
return_type operator ++ ( )  
{  
    //body  
}
```

## Example using unary minus (-) using member function

```
#include <iostream>
using namespace std;
class Number {
private:
    int value;
public:
    // Constructor
    Number(int v = 0) {
        value = v; }
    // Overload unary minus operator
    Number operator-() {
        return Number(-value); // Return a
new Number object with negated value
    }
    // Display method
    void display() {
        cout << "Value: " << value << endl;
    } };
```

```
int main() {
    Number num1(10);

    cout << "Original Number: ";
    num1.display(); // Output: Value: 10

    Number num2 = -num1; // Using overloaded
unary minus operator

    cout << "Negated Number: ";
    num2.display(); // Output: Value: -10

    return 0;
}
```

## Example using unary minus (-) using non-member function

```
#include <iostream>
using namespace std;
class Number {
public:
    int value;
    // Constructor
    Number(int v = 0){
        value = v;
    }
    // Display method
    void display(){
        cout << "Value: " << value << endl;
    }
};
// Non-member function to overload unary minus
operator-(Number& num) {
    return Number(-num.value); // Return a new
    Number object with negated value
}
```

```
int main() {
    Number num1(10);
    cout << "Original Number: ";
    num1.display(); // Output: Value: 10

    // Using overloaded unary minus operator
    as non-member function

    Number num2 = -num1;
    cout << "Negated Number (non-member
function): ";
    num2.display(); // Output: Value: -10
    (original value negated)

    return 0;
}
```

# Overloading Increment operator (++)

When applied to basic data types prefix and postfix operators are two different operators.

But when they are overloaded there is no distinction (difference) between prefix and postfix notation. The expression `c2=c1++` has exactly the same effect as `c2=++c1` .

For both postfix and prefix increment operators, the overloaded function name would be the same (`operator++`). The number arguments would also be the same.

**So how it is possible to differentiate the two when overloading?**

C++ uses a “dummy variable” or “dummy argument” for the postfix operators.

To overload the postfix operator, C++ uses a fake integer parameter that only serves to distinguish the postfix version of increment/decrement from the prefix version.

If we need the basic data type increment for the overloaded function, then it requires two overloaded operators, one for prefix and one for post fix.

```
operator ++ ( ); //prefix  
operator ++ (int ); //post fix
```

The second declaration uses a dummy ‘int’ argument, which is set to zero automatically by the post fix (++) operator. This extra argument allows the compiler to distinguish the two forms.

```

#include <iostream>
using namespace std;
class Count {
private:
    int value;
public:
    // Constructor to initialize count to 5
    Count(){ value = 5; }

    // Overload ++ when used as postfix
    void operator++(int) {
        // Postfix increment
        value++;
    }
    // Overload ++ when used as prefix
    void operator++() {
        // Prefix increment
        ++value;
    }
    void display() {
        cout << "Count: " << value << endl;
    }
};

```

```

int main() {
    Count count1;

    // Postfix increment operator
    count1++; // Equivalent to
count1.operator++(0)
    count1.display(); // Output: Count: 6

    // Prefix increment operator
    ++count1; // Equivalent to
count1.operator++()
    count1.display(); // Output: Count: 7

    return 0;
}

```

# Overloading Binary Operators

Binary operators can be overloaded just as easily as unary operators.

Binary operators act on two operands.

Examples:

Arithmetic operators (+, -, \*, /, %), relational operators (==, >, < etc.),  
assignment operator (=).

# Binary Operator Overloading using Member Function

```
int a=10, b=5;  
int c = a + b;
```

where **a** is known as left-hand operand as it is in the left side of operator (+) and **b** is known as right hand operand as it is right side of operator (+).

In binary operator overloading using **member function**, we need to define operator function inside class with **right hand** operand as an argument to it.

The left hand operand must be the object of class where an operator function is defined.

Syntax:

```
return_type operator op (class_name &object)  
{  
function body;  
}
```

## Binary Operator Overloading using Member Function

In overloading binary operators the left hand operand is used to invoke the operator function and the right operand is passed as an argument.

e.g: **s3 = s1 + s2;** // s1, s2, s3 are objects of a class with overloaded

In this example the left hand operand invokes operator + ( ) with right hand operand as an argument.

Hence the above statement **s3=s1+s2** is equivalent to **s3= s1.operator + (s2).**

## Example with binary plus (+) using member function

```
#include <iostream>
using namespace std;
class Number {
private:
    int value;
public:
    // Constructor
    Number(int v = 0){
        value = v;
    }
    // Overload binary + operator using member
    function
    Number operator+(Number &n) {
        return Number(value + n.value); // Return
    a new Number object with sum of values
    }
    // Display method
    void display(){
        cout << "Value: " << value << endl;
    } };

```

```
int main() {
    Number num1(5);
    Number num2(10);
    // Using overloaded + operator as member
    function
    Number num3 = num1 + num2;

    cout << "num1: ";
    num1.display(); // Output: Value: 5

    cout << "num2: ";
    num2.display(); // Output: Value: 10

    cout << "Result of num1 + num2: ";
    num3.display(); // Output: Value: 15

    return 0;
}

```

# Overloading operators using friend function

## Friend Function in C++

A **friend function** is a function that is not a member of a class but has access to its private and protected members. This allows the function to perform operations that would typically require member functions, without being part of the class itself.

### Key Points about Friend Functions:

1. **Access Private Members:** A friend function can access the private and protected members of the class as if it were a member.
2. **Declaration:** To declare a friend function, use the **friend** keyword in the class where the function needs access.
3. **Types of Friend Functions:** A friend function can be a normal function or a member function of another class.
4. **Friend Keyword:** The **friend** keyword is placed only in the function declaration inside the class, not in the function definition.

# Operator Overloading Using Friend Function

When overloading operators using a friend function:

1. **Unary Operators:** The function requires one argument.
2. **Binary Operators:** The function requires two arguments.
3. **Flexibility:** Allows built-in data types as the left operand, unlike member function overloading, where the left operand must be an object of the class.

For example, consider the statement `c = 2 + c1;`. This works for a friend function but not for a member function.

# Syntax

```
// Unary operator overload
```

```
friend returnType operator opr_symbol (ClassName& obj);
```

```
// Binary operator overload
```

```
friend returnType operator opr_symbol (ClassName& obj1, ClassName& obj2);
```

*Opr\_symbol* = operator symbol to be overloaded

# Overload Unary Minus (-) Operator Using a Friend Function

```
#include <iostream>
using namespace std;
class Number {
private:
    int value;
public:
    // Constructor
    Number(int v = 0){
        value = v;
    }
    // Friend function to overload unary minus
    operator
    friend Number operator-(Number& n);

    // Display method
    void display(){
        cout << "Value: " << value << endl;
    }
};
```

```
// Definition of the friend function
Number operator-(Number& n) {
    return Number(-n.value); // Return a new
    Number object with negated value
}

int main() {
    Number num1(10);

    cout << "Original Number: ";
    num1.display(); // Output: Value: 10

    Number num2 = -num1; // Using overloaded
    unary minus operator
    cout << "Negated Number: ";
    num2.display(); // Output: Value: -10

    return 0;
}
```

# Overloading the binary (+) Operator Using a Friend Function

```
#include <iostream>
using namespace std;
class Number {
private:
    int value;
public:
    // Constructor
    Number(int v = 0) {
        value = v;
    }

    // Friend function to overload + operator
    friend Number operator+(Number& n1, Number& n2);
    // Display method
    void display() {
        cout << "Value: " << value << endl;
    }
};
```

```
// Definition of the friend function
Number operator+(Number& n1, Number& n2) {
    return Number(n1.value + n2.value); // Return
    a new Number object with sum of values
}

int main() {
    Number num1(5);
    Number num2(10);
    // Using overloaded + operator as friend
    function
    Number result = num1 + num2;
    cout << "num1: ";
    num1.display(); // Output: Value: 5
    cout << "num2: ";
    num2.display(); // Output: Value: 10
    cout << "Result of num1 + num2: ";
    result.display(); // Output: Value: 15
    return 0; }
```

# Overloading I/O operators

Q. Can we directly use Input/Output operators on objects ?

**NO!! Why?**

**Operators Predefined for Built-in Types:**

- The `>>` and `<<` operators are predefined to work with built-in data types like `int`, `float`, `char`, etc.

**User-Defined Types:**

- Classes and objects are user-defined types, not built-in types.

**Compiler Error:**

- Trying to use `>>` or `<<` with objects of user-defined types directly will result in a compiler error.

## Example

```
int a;  
cin>>a;  
cout<<a<<endl;
```

here, Input/Output Operators (>>/<<) can be used directly as 'a' is a built-in data type (int).

```
class C{  
  
};  
int main()  
{  
    C c1;  
    cin>>c1;  
    cout<<c1;  
    return 0;  
}
```

c1 is an object of type "class C". Here compiler will generate an error as we are trying to use Input/Output Operators (>>/<<) on user-defined data types.

```

#include <iostream>
using namespace std;
class Complex {
private:
    int real;
    int imag;
public:
    // Constructor
    Complex(int r = 0, int i = 0){
        real = r;
        imag = i;
    }
    // Friend function to overload the << operator
    friend ostream& operator<<(ostream& out, Complex& c);
    // Friend function to overload the >> operator
    friend istream& operator>>(istream& in, Complex& c);
};
// Definition of the overloaded << operator
ostream& operator<<(ostream& out, Complex& c) {
    out << c.real << " + " << c.imag << "i";
    return out;
}

```

```

// Definition of the overloaded >> operator
istream& operator>>(istream& in, Complex& c) {
    cout << "Enter real part: ";
    in >> c.real;
    cout << "Enter imaginary part: ";
    in >> c.imag;
    return in;
}

int main() {
    Complex c1;

    // Use overloaded >> operator to input a complex
    number
    cin >> c1;

    // Use overloaded << operator to output a complex
    number
    cout << "The complex number is: " << c1 << endl;

    return 0;
}

```

# Overloading functions

```
// Friend function to overload the << operator
friend ostream& operator<<(ostream& out, Complex& c);

// Friend function to overload the >> operator
friend istream& operator>>(istream& in, Complex& c);
```

In C++, overloading the << and >> operators as member functions is generally not possible because the left-hand operand (usually `std::ostream` or `std::istream`) is not an instance of the class being defined. Instead, these operators are typically overloaded as friend functions or regular non-member functions.

Here, it is important to make operator overloading function a friend of the class because it can access private members of the class and would be called without creating an object.

# Limitations of operator overloading

- Not all operators can be overloaded (e.g., `::`, `.`, `.*`, `?:`, `sizeof`).
- Only existing operators can be overloaded. New operators cannot be created or existing operators renamed.
- At least one operand in an overloaded operator must be a user-defined type (e.g., a class or struct).
- The number of operands an operator takes cannot be changed. Binary operators must remain binary, and unary operators must remain unary.
- The default precedence and associativity of operators cannot be altered. Even if an operator is overloaded, its precedence and associativity in expressions remain unchanged.



Any Queries?